

Sprawozdanie

Nr laboratorium: 01

Temat: Arytmetyka komputerowa

Autor: Maciej Bielewski

Data: 01/10/2026

1. Co było do zrobienia

Celem laboratorium było zbadanie, jak istotną rolę w obliczeniach komputerowych odgrywa precyzja, z jaką liczby są reprezentowane w komputerze, oraz wyznaczenie wartości epsilon maszynowego dla różnych precyzji. W ramach zadania należało również zaobserwować wpływ sposobu wykonywania obliczeń na otrzymywane wyniki.

W pierwszej części należało napisać program wyznaczający kolejne wyrazy ciągu:

$$x_{n+1} = x_n + 3x_n(1 - x_n),$$

dla wartości początkowej:

$$x_0 = 0.01.$$

Obliczenia należało wykonać dla dwóch typów danych: `float` oraz `double`, a następnie porównać otrzymane wyniki i wyjaśnić przyczyny ich rozbieżności.

W drugiej części należało obliczyć ten sam ciąg przy użyciu algebraicznie równoważnej postaci:

$$x_{n+1} = 4x_n - 3x_n^2,$$

a następnie porównać wyniki uzyskane przy użyciu obu wzorów.

Ostatnim zadaniem było eksperymentalne wyznaczenie epsilon maszynowego dla typów `float` i `double`, czyli wartości charakteryzującej dokładność reprezentacji liczb zmiennoprzecinkowych w otoczeniu liczby 1.

2. Moje podejście do rozwiązania problemu

Do rozwiązania zadania wykorzystano język C. Wybrano go ze względu na możliwość bezpośredniego wykorzystania typów `float` i `double` oraz kontrolowania typu wykonywanych operacji zmiennoprzecinkowych.

2.1. Obliczanie pierwszego wzoru

Dla pierwszej postaci ciągu zastosowano wzór:

$$x_{n+1} = x_n + 3x_n(1 - x_n),$$

dla wartości początkowej $x_0 = 0.01$.

Obliczenia wykonywano równolegle dla typów `float` i `double`. Dla wartości typu `float` zastosowano sufix `f`, np. `0.01f`, `3.0f` oraz `1.0f`. Dzięki temu odpowiednie operacje są wykonywane z wykorzystaniem wartości typu `float`.

Typ `float` zapewnia mniejszą precyzję reprezentacji niż `double`. Dla typowych reprezentacji IEEE 754 odpowiada to odpowiednio około 24 i 53 bitom precyzji znaczącej. Mniejsza precyzja powoduje większe błędy zaokrągleń podczas wykonywania operacji.

W każdej iteracji zapisywano aktualną wartość ciągu dla obu typów oraz obliczano bezwzględną różnicę:

$$d_n = |x_n^{float} - x_n^{double}|.$$

Pozwoliło to obserwować, jak wraz z kolejnymi iteracjami zmieniają się wyniki uzyskiwane dla obu reprezentacji.

W obliczaniu różnicy wartość `float` jest konwertowana do `double`. Konwersja ta nie zwiększa dokładności wcześniej obliczonej wartości `float`, a jedynie umożliwia wykonanie porównania w jednym typie.

2.2. Obliczanie drugiego wzoru

Następnie zastosowano drugą postać wzoru:

$$x_{n+1} = 4x_n - 3x_n^2.$$

Jest ona algebraicznie równoważna pierwszemu wzorowi:

$$\begin{aligned} x_n + 3x_n(1 - x_n) &= x_n + 3x_n - 3x_n^2 \\ &= 4x_n - 3x_n^2. \end{aligned}$$

Mimo równoważności matematycznej oba wzory mogą dawać różne wyniki w obliczeniach komputerowych. Wynika to z tego, że wymagają wykonania innego zestawu operacji pośrednich, przez co zaokrąglenia mogą występować w różnych miejscach.

Drugi wzór również obliczano osobno dla `float` i `double`, a następnie porównywano otrzymane wyniki z rezultatami pierwszej metody.

2.3. Wyznaczenie epsilon maszynowego

Epsilon maszynowy wyznaczono eksperymentalnie osobno dla `float` i `double`. Wartość początkową przyjęto jako:

$$a = 1.$$

Następnie wartość a była zmniejszana dwukrotnie:

$$a \leftarrow \frac{a}{2}.$$

W każdej iteracji sprawdzano, czy nadal zachodzi:

$$1 + \frac{a}{2} > 1.$$

Jeżeli warunek był spełniony, wartość zmniejszano ponownie. Procedurę kończono, gdy dalsze zmniejszenie wartości nie powodowało już zmiany reprezentowanej liczby 1. Otrzymana wartość odpowiada konwencjonalnie definiowanemu epsilonowi maszynowemu dla danej reprezentacji.

2.4. Kod programu

Poniżej przedstawiono kompletny kod programu wykorzystanego do wykonania obliczeń:

```
#include <stdio.h>
#include <float.h>
#include <math.h>

void sequence(int N){
    float xf = 0.01f;
    double xd = 0.01;

    printf("\nFormuła 1: x(n+1) = x(n) + 3*x(n)*(1-x(n))\n");
    printf("%5s %15s %20s %20s\n", "n", "float", "double", "difference");
    for (int n = 0; n <= N; n++) {
        printf("%5d %15.10f %20.10f %20.7e\n",
            n, xf, xd, fabs((double)xf - xd));
        xf = xf + 3.0f * xf * (1.0f - xf);
        xd = xd + 3.0 * xd * (1.0 - xd);
    }
}

void sequence2(int N){
    float xf = 0.01f;
    double xd = 0.01;

    printf("\nFormuła 2: x(n+1) = 4*x(n) - 3*x(n)^2\n");
```

```
printf("%5s %15s %20s %20s\n", "n", "float", "double", "difference");
for (int n = 0; n <= N; n++) {
    printf("%5d %15.10f %20.10f %20.7e\n",
        n, xf, xd, fabs((double)xf - xd));
    xf = 4.0f * xf - 3.0f * xf * xf;
    xd = 4.0 * xd - 3.0 * xd * xd;
}

}

void machine_epsilon(void) {
    float ef = 1.0f;
    double ed = 1.0;

    while (1.0f + ef / 2.0f > 1.0f)
        ef /= 2.0f;

    while (1.0 + ed / 2.0 > 1.0)
        ed /= 2.0;

    printf("\nMachine epsilon\n");
    printf("float: %.9g\n", ef);
    printf("double: %.17g\n", ed);
}

int main(){
    sequence(50);
    sequence2(50);
    machine_epsilon();
    return 0;
}
```

3. Wyniki

Obliczenia wykonano dla 50 iteracji, czyli dla wartości n od 0 do 50. Poniższa tabela zawiera wyniki otrzymane dla obu wzorów oraz dla typów float i double. Ostatnie dwie kolumny przedstawiają bezwzględną różnicę pomiędzy wynikami uzyskanymi dla obu typów danych.

n	Wzór 1 float	Wzór 1 double	Wzór 2 float	Wzór 2 double	Wzór 1 różnica	Wzór 2 różnica
0	0.0099999998	0.0100000000	0.0099999998	0.0100000000	2.2351742e-10	2.2351742e-10
1	0.0397000015	0.0397000000	0.0396999978	0.0397000000	1.4781952e-09	2.2470951e-09
2	0.1540717334	0.1540717300	0.1540717185	0.1540717300	3.3555221e-09	1.1545639e-08
3	0.5450726151	0.5450726260	0.5450726151	0.5450726260	1.0897784e-08	1.0897784e-08
4	1.2889780998	1.2889780012	1.2889779806	1.2889780012	9.8634197e-08	2.0575092e-08
5	0.1715188026	0.1715191421	0.1715192795	0.1715191421	3.3946635e-07	1.3737081e-07
6	0.5978190899	0.5978201201	0.5978205204	0.5978201201	1.0302176e-06	4.0029390e-07
7	1.3191133738	1.3191137924	1.3191139698	1.3191137924	4.1865739e-07	1.7738906e-07
8	0.0562732220	0.0562715776	0.0562710762	0.0562715776	1.6443233e-06	5.0144387e-07
9	0.2155928612	0.2155868392	0.2155850083	0.2155868392	6.0219429e-06	1.8309690e-06
10	0.7229306102	0.7229143012	0.7229093313	0.7229143012	1.6309000e-05	4.9698579e-06
11	1.3238364458	1.3238419442	1.3238437176	1.3238419442	5.4983600e-06	1.7734066e-06
12	0.0377169847	0.0376952973	0.0376882553	0.0376952973	2.1687494e-05	7.0419447e-06
13	0.1466002166	0.1465183827	0.1464918107	0.1465183827	8.1833914e-05	2.6572034e-05
14	0.5219259858	0.5216706214	0.5215876698	0.5216706214	2.5536438e-04	8.2951586e-05
15	1.2704837322	1.2702617739	1.2701895237	1.2702617739	2.2195829e-04	7.2250238e-05
16	0.2395482063	0.2403521728	0.2406139374	0.2403521728	8.0396645e-04	2.6176460e-04
17	0.7860428095	0.7881011902	0.7887705564	0.7881011902	2.0583807e-03	6.6936622e-04
18	1.2905813456	1.2890943028	1.2886053324	1.2890943028	1.4870428e-03	4.8897042e-04

n	Wzór 1 float	Wzór 1 double	Wzór 2 float	Wzór 2 double	Wzór 1 różnica	Wzór 2 różnica
19	0.1655247211	0.1710848467	0.1729102135	0.1710848467	5.5601256e-03	1.8253668e-03
20	0.5799036026	0.5965293125	0.6019470096	0.5965293125	1.6625710e-02	5.4176971e-03
21	1.3107497692	1.3185755880	1.3207675219	1.3185755880	7.8258188e-03	2.1919339e-03
22	0.0888042450	0.0583776083	0.0497894287	0.0583776083	3.0426637e-02	8.5881796e-03
23	0.3315584064	0.2232865976	0.1917207539	0.2232865977	1.0827181e-01	3.1565844e-02
24	0.9964407086	0.7435756764	0.6566124558	0.7435756767	2.5286503e-01	8.6963221e-02
25	1.0070805550	1.3155883460	1.3330299854	1.3155883459	3.0850779e-01	1.7441640e-02
26	0.9856885076	0.0700352956	0.0012130737	0.0700352962	9.1565321e-01	6.8822222e-02
27	1.0280085802	0.2654263545	0.0048478805	0.2654263566	7.6258223e-01	2.6057848e-01
28	0.9416294098	0.8503519691	0.0193210151	0.8503519740	9.1277441e-02	8.3103096e-01
29	1.1065198183	1.2321124624	0.0761641562	1.2321124569	1.2559264e-01	1.1559483e+00
30	0.7529209256	0.3741464896	0.2872536778	0.3741465083	3.7877444e-01	8.6892830e-02
31	1.3110139370	1.0766291714	0.9014706612	1.0766292041	2.3438477e-01	1.7515854e-01
32	0.0877830982	0.8291255674	1.1679346561	0.8291254870	7.4134247e-01	3.3880917e-01
33	0.3280147910	1.2541546501	0.5795245171	1.2541547284	9.2613986e-01	6.7463021e-01
34	0.9892780781	0.2979069415	1.3105521202	0.2979066652	6.9137114e-01	1.0126455e+00
35	1.0210989714	0.9253821286	0.0895681381	0.9253815174	9.5716843e-02	8.3581338e-01
36	0.9564665556	1.1325322627	0.3342052102	1.1325332114	1.7606571e-01	7.9832800e-01
37	1.0813814402	0.6822410727	1.0017414093	0.6822384208	3.9914037e-01	3.1950299e-01
38	0.8173682690	1.3326056470	0.9965081215	1.3326058948	5.1523738e-01	3.3609777e-01
39	1.2652003765	0.0029091569	1.0069472790	0.0029081668	1.2622912e+00	1.0040391e+00
40	0.2586054802	0.0116112380	0.9859607220	0.0116072949	2.4699424e-01	9.7435343e-01
41	0.8337915540	0.0460404896	1.0274872780	0.0460249919	7.8775106e-01	9.8146229e-01
42	1.2495411634	0.1778027783	0.9427587986	0.1777450679	1.0717384e+00	7.6501373e-01
43	0.3141053319	0.6163696291	1.1046526432	0.6162003443	3.0226430e-01	4.8845230e-01
44	0.9604348540	1.3257439574	0.7578382492	1.3256927842	3.6530910e-01	5.6785454e-01
45	1.0744340420	0.0301847079	1.3083965778	0.0303870624	1.0442493e+00	1.2780095e+00
46	0.8345106244	0.1180054819	0.0978813171	0.1187781289	7.1650514e-01	2.0896812e-02
47	1.2488185167	0.4302460463	0.3627830148	0.4327877839	8.1857247e-01	7.0004769e-02
48	0.3166309595	1.1656492041	1.0562975407	1.1692353380	8.4901824e-01	1.1293780e-01
49	0.9657583237	0.5863826153	0.8778967857	0.5756075252	3.7937571e-01	3.0228926e-01
50	1.0649658442	1.3139967466	1.1994788647	1.3084580316	2.4903090e-01	1.0897917e-01

Już od pierwszych iteracji pojawiają się niewielkie różnice pomiędzy float i double . Dla pierwszego wzoru różnica wynosi $2.24 \cdot 10^{-10}$ dla $n = 0$, $1.63 \cdot 10^{-5}$ dla $n = 10$ oraz $2.53 \cdot 10^{-1}$ dla $n = 24$. W dalszych iteracjach rozbieżność staje się jeszcze większa i dla $n = 39$ osiąga około 1.26.

Podobne zjawisko występuje dla drugiego wzoru. Przykładowo, dla $n = 29$ różnica pomiędzy float i double wynosi około 1.16, a dla $n = 45$ około 1.28.

Widoczny jest również wpływ postaci wzoru na wyniki. Dla $n = 24$ wartości double są niemal identyczne:

$$0.7435756764 \quad \text{oraz} \quad 0.7435756767,$$

podczas gdy dla float wynoszą:

$$0.9964407086 \quad \text{oraz} \quad 0.6566124558.$$

Otrzymane wyniki pokazują więc, że zarówno mniejsza precyzja float , jak i zmiana postaci wzoru mogą prowadzić do znacznych różnic po wykonaniu wielu iteracji.

Wartości epsilon maszynowego wyznaczone przez program wyniosły:

Typ	Epsilon maszynowy
float	$1.1920929 \cdot 10^{-7}$
double	$2.2204460492503131 \cdot 10^{-16}$

Wynik potwierdza znacznie większą precyzję typu `double` w otoczeniu liczby 1.

4. Weryfikacja poprawności rozwiązania

Poprawność rozwiązania zweryfikowano poprzez sprawdzenie zależności matematycznych, porównanie wybranych wyników z obliczeniami ręcznymi oraz odniesienie otrzymanych wartości epsilon maszynowego do typowych reprezentacji IEEE 754.

4.1. Weryfikacja epsilon maszynowego

Dla typowych reprezentacji IEEE 754 wartości teoretyczne epsilon maszynowego wynoszą:

$$\begin{aligned}\varepsilon_{float} &= 2^{-23} \approx 1.1920928955 \cdot 10^{-7}, \\ \varepsilon_{double} &= 2^{-52} \approx 2.220446049250313 \cdot 10^{-16}.\end{aligned}$$

Program wyznaczył odpowiednio:

$$\begin{aligned}\varepsilon_{float} &= 1.1920929 \cdot 10^{-7}, \\ \varepsilon_{double} &= 2.220446049250313 \cdot 10^{-16}.\end{aligned}$$

Wyniki zgadzają się z wartościami teoretycznymi do dokładności prezentowanej przez program. Potwierdza to poprawność zastosowanej procedury wyznaczania epsilon.

4.2. Weryfikacja wzorów ciągu

Oba zastosowane wzory są algebraicznie równoważne:

$$x_n + 3x_n(1 - x_n) = 4x_n - 3x_n^2.$$

Dla wartości początkowej $x_0 = 0.01$ otrzymujemy w obu przypadkach:

$$x_1 = 0.01 + 3 \cdot 0.01(1 - 0.01) = 0.0397,$$

oraz:

$$x_1 = 4 \cdot 0.01 - 3 \cdot 0.01^2 = 0.0397.$$

Wyniki uzyskane przez program są bardzo bliskie tej wartości, co potwierdza poprawność implementacji pierwszej iteracji.

4.3. Weryfikacja rozbieżności wyników

Rozbieżności pomiędzy wynikami obu wzorów nie wskazują na błąd implementacji, lecz wynikają z właściwości arytmetyki zmiennoprzecinkowej. Matematycznie równoważne wyrażenia mogą prowadzić do różnych wyników numerycznych, ponieważ wykorzystują inne operacje pośrednie, a zaokrąglenia mogą występować w różnych miejscach.

Dla przykładu, przy $n = 24$ wyniki dla `double` wynoszą:

$$0.7435756764 \quad \text{oraz} \quad 0.7435756767,$$

czyli różnią się bardzo nieznacznie. Dla `float` otrzymano natomiast:

$$0.9964407086 \quad \text{oraz} \quad 0.6566124558.$$

Ponieważ każdy kolejny wyraz ciągu jest obliczany na podstawie poprzedniego, błędy zaokrągleń są propagowane do kolejnych iteracji. Zgodnie z podejściem stosowanym w analizie błędów prowadzi to do stopniowego zwiększania różnic pomiędzy obliczanymi ciągami.

5. Wnioski

Przeprowadzone eksperymenty pokazały, że precyzja reprezentacji liczb oraz sposób wykonywania operacji mają istotny wpływ na wyniki obliczeń numerycznych.

1. Różnica precyzji typów danych.

Typ `double` zapewnia większą precyzję reprezentacji niż `float`, co znajduje potwierdzenie w wartościach epsilon maszynowego. Mniejsza precyzja powoduje większe błędy zaokrągleń, które w kolejnych iteracjach prowadzą do coraz większych różnic w wynikach.

2. Wpływ postaci wzoru na wynik obliczeń.

Wzory

$$x_n + 3x_n(1 - x_n)$$

oraz

$$4x_n - 3x_n^2$$

są matematycznie równoważne, jednak mogą dawać różne wyniki w arytmetyce zmiennoprzecinkowej. Wynika to z różnej kolejności wykonywania operacji oraz zaokrągleń wartości pośrednich. Efekt ten jest szczególnie widoczny dla typu `float`.

3. Propagacja błędów w kolejnych iteracjach.

Każdy kolejny wyraz ciągu zależy od poprzedniego, dlatego błąd powstały w jednej iteracji wpływa na następne wartości. W rezultacie początkowo bardzo małe różnice mogą po wielu iteracjach urosnąć do wartości rzędu 1.

Przeprowadzone doświadczenie pokazuje więc, że w obliczeniach numerycznych poprawność matematyczna wzoru nie gwarantuje identycznych wyników uzyskanych komputerowo. Na wynik wpływają również precyzja reprezentacji liczb, sposób wykonywania operacji oraz propagacja błędów zaokrągleń.